

CompanyGraph – Von einem Befehl zur vollständigen Integration

Wie weit das Modell einer Firma reicht, Stufe für Stufe.

BERUHT AUF `concept Instance`

Robert Blust · Software Engineer & Architect

Ein Modell, fünf Stufen

Modell. Geprüft. Geregelt. Bereitgestellt. Veröffentlicht. Jede Stufe fügt genau eine Sache hinzu, das aus dem Modell liest, und keine verlangt, dass Sie es noch einmal schreiben.

BERUHT AUF

concept Instance

concept Core

5 Veröffentlicht

4 Bereitgestellt

3 Geregelt

2 Geprüft

1 Modell

Stufe 1 – Modell: ein Befehl

5 Veröffentlicht

4 Bereitgestellt

3 Geregelt

2 Geprüft

1 Modell

`init` schreibt die Instanz: den core aus einem benannten Release, ein Manifest, das ihn Datei für Datei festhält, die Prüfung, die Agent-Skills und einen Commit-Hook.

Das Modell ist ein Ordner mit Markdown. Für den Anfang braucht es nichts weiter.

BERUHT AUF

feature An instance is made and kept from one command

concept Pin

decision Core is vendored into an instance at a release its manifest names

feature An agent does the work no script can, by the skills an instance ships

```
$ npx --yes 'github:companygraph/meta-model#semver:*' init acme
✓ files written into acme
  core <release>, vendored under meta/core/
  the model is empty but for its README files, its source and its
  singular entities
  run "npx github:companygraph/meta-model#<release> check acme"
  whenever it changes
```

Stufe 2 – Geprüft: Jede Änderung wird geprüft

Liegt der Ordner in einem Repository, führt sein Workflow bei jedem Pull-Request dieselbe Prüfung aus, mit dem Release, das das Manifest nennt. Ist der Branch main geschützt, gelangt nie ein Verweis ins Leere dorthin.

5 Veröffentlicht

4 Bereitgestellt

3 Geregelt

2 Geprüft

1 Modell

BERUHT AUF `feature Checks an instance runs` `concept Check`

question `Will a new CompanyGraph release break our model?`

companygraph / companygraph ✓

conventions / conventions ✓

Stufe 3 – Geregelt: Ihre Regeln, unsere Werkzeuge

5 Veröffentlicht

4 Bereitgestellt

3 Geregelt

2 Geprüft

1 Modell

Jedes Repository rund um das Modell wird an einer einheitlichen Markdown-Form gemessen. Es deklariert in `pins.json`, was es von einem anderen übernimmt, und erfährt, welche dieser Pins im Rückstand sind. Ein Repository ohne Modell bekommt dieselben Werkzeuge mit `adopt`.

Es sind unsere Werkzeuge, nicht unsere Regeln: Ihre Sitze, Ihre Repositorys, Ihre Konventionen.

BERUHT AUF

feature A repository is held to one form and told which pins are behind

concept Pin

concept Check

decision An agent's commit is authored by the seat it held

```
$ companygraph pins
Pins of acme
  current core-release companygraph/meta-model in
  .companygraph/manifest.json: <release>
```

Auch Regeln, Risiken und Kontrollen sind Seiten

Eine Regel sagt, was geschehen muss, nicht geschehen darf oder geschehen darf, und warum. Ein Risiko sagt, was schiefgehen könnte. Eine Kontrolle sagt, wodurch sich das verhindern lässt, und verweist auf die Prüfung, die das tut.

Die Kontrolle ist die Seite, die Prüfung bleibt Code.

BERUHT AUF

feature Checks an instance runs

concept Check

```
# Main requires a review
kind: preventive   mode: automated
enforces: A change is reviewed before it ships
mitigates: An unreviewed change reaches customers
```

Stufe 4 – Bereitgestellt: Ein Agent fragt das Modell

Ein nur lesender Server antwortet einem Agenten über MCP, und ein Chat antwortet einem Besucher. Beide antworten aus dem Modell auf dem Stand eines benannten Commits und nennen die Seiten, auf denen jede Antwort beruht.

BERUHT AUF `feature An agent reads the model`

`feature A visitor asks the model`

`decision The server only reads, and answers at one named commit`

`question Can I trust what this chat says?`

```
search "Resolution"
```

```
→ Resolution    bounded-context
```

```
    Turns the names a page writes into the edges of the graph...
```

Stufe 5 – Veröffentlicht: Seiten aus dem Modell

Die Website leitet ihren Graphen, ihre Seiten und deren strukturierte Daten aus dem Modell ab, auf dem Stand eines gepinnten Commits, und der Editor liest dieselben Dateien. Ändern Sie eine Seite, verschieben Sie den Pin, und die Website zeigt die Änderung.

BERUHT AUF `feature The vocabulary on the web`

`feature Checks while a file is edited`

5 **Veröffentlicht**

4 **Bereitgestellt**

3 **Geregelt**

2 **Geprüft**

1 **Modell**

Ein Pack steigt alle Stufen hinauf

Am 2. Oktober 2026 gelangte das Software-Pack von einem Release in unser eigenes Modell, in unseren ersten Bounded Context, auf die Website, in den Server und in den Editor, Schritt für Schritt, jeder mit einem Pin.

BERUHT AUF

concept Pack

bounded-context Resolution



Pins halten die Stufen zusammen

Jede Stufe pinnt die darunter: auf einen Tag für das, was sie ausführt, auf einen Commit für das, was sie zeigt. Ein Pin wird verschoben, wenn jemand das entscheidet.

Am selben Tag hielt ein Pin, den wir nicht verschoben hatten, jede Prüfung in einer Organisation an. Ihn zu verschieben, brachte alle wieder in Ordnung.

BERUHT AUF `concept Pin` `concept Release` `decision A release is its tag, and nothing we make is published to a registry`

Wo stehen Sie?

Die Stufen sind eine Leiter, keine Checkliste. Jede macht sich bezahlt, bevor Sie die nächste nehmen. Beginnen Sie mit einem einzigen Befehl.

BERUHT AUF `feature` An instance is made and kept from one command

